

HTML +

CSS

(WRITTEN NOTES)

-(SHUSHANTH)

HTML → Hypertext Markup language

- * used to create a web page's structure
- * It tells about heading, paragraph, imgs, buttons, forms.

Tags: * It tells how to display content in a browser.

```
<html>
</html>
<body>
</body>
```

} tags

Features:

- * Easy to learn
- * platform independent
- * Supported by all browsers

* HTML5 ఇప్పుడు వాడుతున్నారు

Element: An element includes the opening tag, content, and closing tag.

<p> Welcome </p>

HTML Document Structure:

<!DOCTYPE html>

<html>

<head>

<title> Shushanth's Page </title>

</head>

<body>

<h1> hi Buddy </h1>

</body>

</html>

<!DOCTYPE html> ⇒ declares that document is an HTML5 document
* It is used to tell the browser to render the page in standards mode.

Attributes: * which provides additional information about an element.

Ex:

src = image location
alt = alternative text shown if img cannot be loaded.

Empty tags: * Void Elements
* These do not have closing tags

Ex:

<hr>

<input>
<meta>
<link>

 → line break

<hr> → creates a horizontal line

Headings:

* HTML provides 6 heading tags

<h1> Main Heading </h1>

<h2> Sub Heading </h2>

<h3> Heading </h3>

<h4> Heading </h4>

<h5> Heading </h5>

<h6> Smallest Heading </h6>

Viewport Meta Tag
⇒ Makes webpage responsive on different screen sizes such as mobile phones, tablets and desktops.

Fieldset:

* It is a tag groups related form elements together and draws a border around them.

Syntax:

<fieldset>
<legend> Title </legend>

Form elements

</fieldset>

Title
Form elements

Summary Tag:

- * defines a visible heading for <details> element.
- * clicking the summary expands or collapses the hidden content.

▶ Hello Guys

▶ Hello Guys

* Sai

* Kiran

placeholder:

→ It is an attribute used to display a short hint or example text inside an <input> (or) <text area>.

<input type = "text" placeholder = "enter ur name">

Text Formatting:

Bold
Strong
Italic
Emphasis
underline
Highlight
Delete
mark
Superscript
Subscript

Metadata:

* provides information about HTML document
* It is not displayed on web page but is used by browsers.
* inserted inside head tag.

Links:

→ The anchor (<a>) tag is used to create hyperlinks, allowing users to navigate from one page to another, another section of same page.

Ex: `<a href = "https://www.google.com">Visit Google </a>`
= `<img src = "image.jpg" alt = "Image">`
↓
Specifies the path

⇒ href = specifies the destination

⇒ <a> = creates hyperlink

⇒ target = specifies where to open the destination

→ self

→ blank

→ parent

→ top

Description list:

* used to display terms and their descriptions.

Ex: `<dl>` → Description list
`<dt>` → Description Term
`<dd>` → Description Details

Tables: → used to display data in rows & columns

- Student marks
- Employee details.

Rowspan → merges multiple rows into a single cell.

→ Syntax: `<table>`
`<tr>`

tr → table row
creates row `<th> Heading </th>`
`<th> Heading </th>`

th → Table heading
creates heading cell `<tr>`
`<td> Data </td>`
`<td> Data </td>`

td → Table data
creates data cell `</tr>`
`</table>`

`<caption>` → Adds a title to table

`<thead>` → Groups header rows

`<tbody>` → Groups body rows

`<tfoot>` → Groups footer rows

Img Tag:

→ used to display image on a webpage.

→ It is an empty (void) element.

→ doesn't have closing tag.

Lists:

* used to display group of related items ① ordered list

Ex: Shopping lists, Menu items

② Unordered list

* A list is a collection of related items displayed in an organized way. ③ Description list <dl>

ordered list: displays items in a specific order using numbers or letters.

`<ol>`
`<li> Item 1 </li>`
`<li> Item 2 </li>`
`<li> Item 3 </li>`
`</ol>`

unordered list: * displays items with bullet points.

`<ul>`
`<li> Item </li>`
`</ul>`

Block-level Elements: `<div>`, `<p>`, `<h1>`
Inline: `<span>`, `<a>`, `<strong>`

Colspan → Multiple Columns merges into a single cell.

Semantic Tags: * HTML Tags that clearly describe the meaning and purpose of their content to both browser and developer.
→ meaningful.

Non-Semantic:

* header

* nav

* main

* section

* article

* aside

* footer

* figure

* figcaption

* details

* summary

* time

* mark

* div

* span

Forms: * It is used to collect user input and send it to server for processing.

* HTML form is used to collect user data, such as names, password, emails etc.

Syntax

```
<form>
</form>
```

Text → `<input type="text">`

Password → `<input type="password">`

Radio button → Allows only one option

Gender ☒ Male ☐ Female

`<input type="radio" name="gender">Male`

`<input type="radio" name="gender">Female`

checkbox → Allows multiple selections

`<input type="checkbox"> Java`

`u u u u Python`

`u u u u SQL`

Submit button `<input type="submit">`

Button `<input type="button" value="Click me">`

Label: Associates text with an i/p field.

`<label for="name">Name</label>`

`<input type="text" id="name">`

iframe

 → inline frame

⇒ InlineFrame is used to embed another HTML pages, website, pdf, inside current webpage.

⇒ window inside a webpage.

HTML Entities

* Special Character Codes

* `<` → `<`

* `>` → `>`

* `&` → `&`

* `"` → `"`

HTML Web Storage

→ Web Storage API allows websites to store data in user's browser.

* local storage — Permanent

* session storage — stores data only for current browser tab.

CSS → Cascading style sheets

- * It is a stylesheet language used to style and design HTML web pages.
- * It controls colors, fonts, size, position, layout, Animations, Responsive Design
- * HTML → structure, CSS → attractive

Types of CSS:

- ① Inline CSS → Inline CSS written inside HTML element using style attribute
- ② Internal CSS
- ③ External CSS → Easy, quick styling
→ But not reusable
→ difficult to maintain

Internal CSS:

→ Internal CSS is written inside <style> tag with <head> section
→ Better than Inline CSS

Syntax:

<!DOCTYPE html>

<html>

<head>

<style>

h1 {

color: blue;

}

</style>

</head>

<body>

<h1> Hello Divya & Sandy </h1>

</body>

</html>

Selectors: * It is used to select HTML elements so that CSS styles can be applied to them.

Selector
Property
Value
color: red;

Types of Selectors:

1. Element Selector:

* selects HTML elements by their tagname.

Example: tagname {

property: value;

}

4. Universal Selector:

→ The universal selector selects every element on webpage.

Example: * {
property: value;
}

* {
margin: 0;
padding: 0;
}

5. Group Selector:

→ multiple selectors with same CSS rules.

h1, p, h2 {
color: red;
}

External CSS:

→ External CSS is written in a separate .css file and linked to HTML file.

index.html

<!DOCTYPE html>

<html>

<head>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h1> Hello Divya & Sandy </h1>

</body>

</html>

style.css

h1 {

color: green;

font-size: 40px;

}

6. Attribute Selector:

→ selects elements based on an attribute.

Example: <body>
html: <input type="text">
<input type="password">
</body>
css: input[type="text"] {
background-color: yellow;
}

7. Descendant Selector:

selects elements inside another element, no matter how deeply nested.

Example: <div>
<p> Hello </p>
</div>
div p {
color: red;
}

→ HTML

→ CSS

heading {
color: green;
font-size: 40px;
}

3. Class Selector:

selects one or more elements using class attribute.

Example: <h1 class="title"> Welcome </h1>
<p class="title"> heading CSS </p>

.title {
color: blue;
}

.title2 {
color: red;
}

Reusable
Lower specificity

unique
Higher specificity

8. child selectors: selects only direct children.

For HTML:

```
<div>
  <p> Paragraph </p>
  <section>
    <p> Another Paragraph </p>
  </section>
</div>
```

CSS:

```
div > p {
  color: blue;
}
```

Backgrounds: The CSS properties (Background) are used to control background appearance of an HTML element.

You can set:

- * Bg Color
- * Bg image
- * Bg repeat
- * Bg position
- * Bg size
- * Bg attachment
- * Multiple bg images.

Bg-repeat: image repeats horizontally and vertically.

Bg-repeat-x:  Bg-repeat-y: 

Bg-no-repeat: no repeats

Background-position: where background image is placed.

- * centre
- * top
- * bottom
- * left / top left
- * right / top right

BORDERS:

→ A border is a line drawn around an HTML element.

- * Highlight element
- * Separate content

h1 {
border: 2px solid red;
}

→ border style

- * value
- * solid
- * dashed
- * dotted
- * double
- * groove
- * none
- * hidden

PADDING: Padding is space b/w content & border of an element.

→ creates space inside an element

```
div {
  padding: 20px;
}
```

MARGINS: A margin is space outside an element's border.

→ It creates distance b/w one element and another.

```
h1 {
  margin: 30px;
}
```

→ top, bottom, right, left

→ Margin-collapse: when 2 vertical

margins touch, only larger margin is applied.

→ Negative margin can also given.

BOX-SIZING:

1. Content-Box
2. Border-Box

Content box: default box-sizing value where the specified width and height apply only to the content; padding & border are added outside.

Border-box: It is box-sizing value where the specified width and height include the content, padding and border.

width = Content only → Content Box

width = Entire Box (Content + Padding + Border) → Border Box

Colors: CSS Colors are used to change the color of: * Text, * Background, * Borders, * shadows, * Gradients.

→ The webpage becomes attractive and easier to read.

COMMON WAYS: 1. Color Names ⇒ color names

2. HEX → Hexadecimal color code

3. RGB

4. RGBA

5. HSL

6. HSLA

(RED GREEN BLUE)

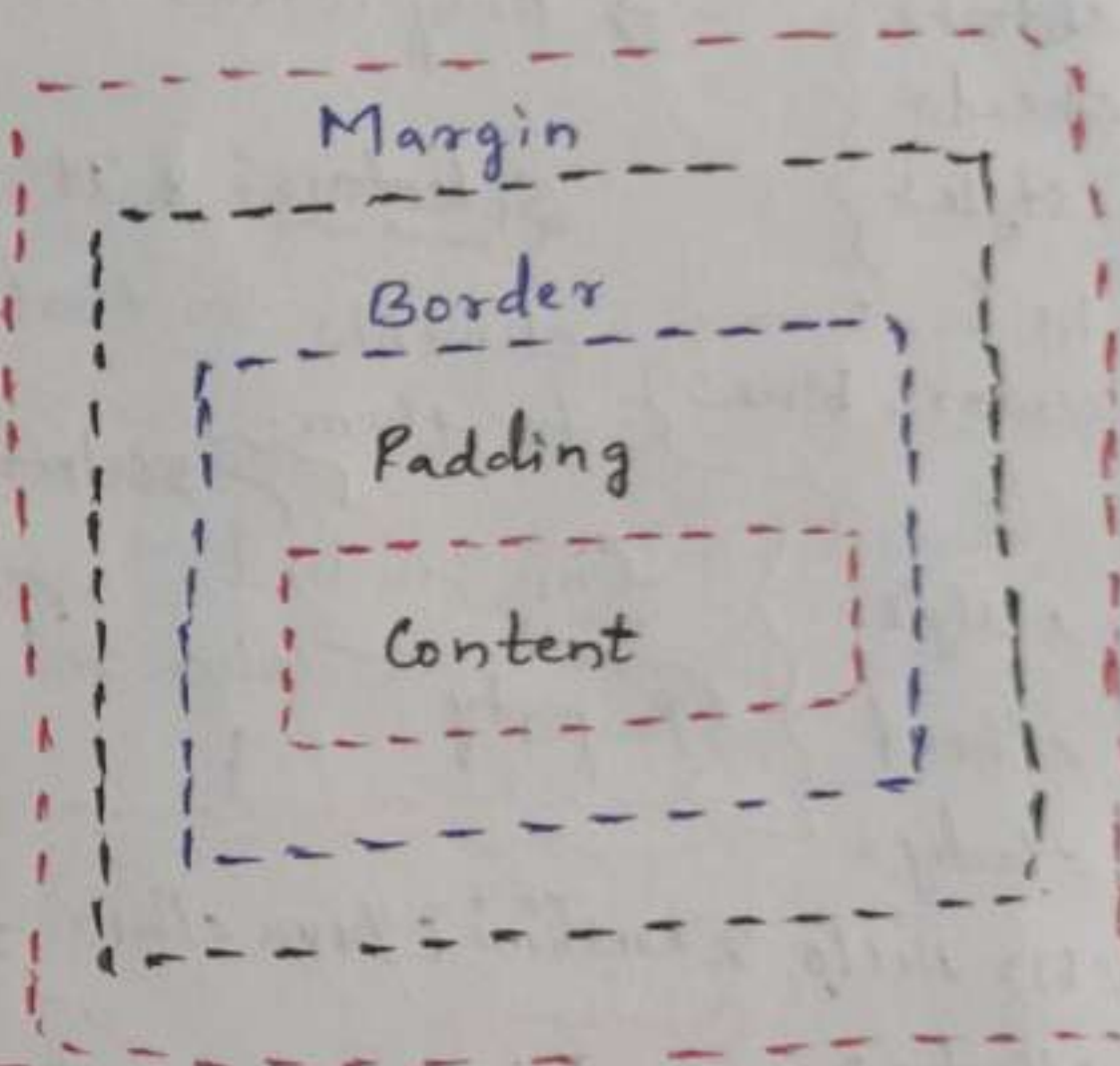
Color: rgb(255, 0, 0);

Red Green Blue Alpha (transparency)

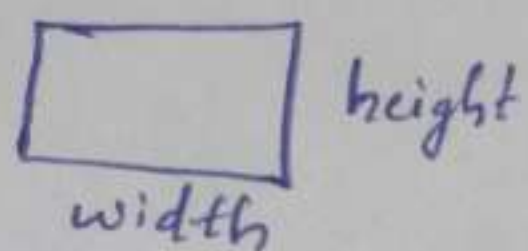
Lightly visible.

Color: rgba(255, 0, 0, 0.5)

opacity: opacity controls transparency of element



Width } → Box Creation
Height }



Display properties: The display property is one of the most important concept.
→ It controls how an element is displayed on a webpage.

Types of Display:

1. Block
2. Inline
3. Inline-Block

① Block: An block element always starts on a new line & takes full available width.
Ex: `<div>`, `<h1>`, `<p>`, `<section>`, `<article>`

```
• box {  
  display: block;  
}
```

② Inline: Inline elements don't start on a new line.

→ They occupy only space needed.

Ex: `<span>`, `<a>`, `<strong>`, `<em>`

```
• box {  
  display: inline;  
}
```

→ Inline elements ignores width & height

③ Inline-Block: Combines the advantages of inline and block.

→ stays in one line

→ allows width & height

```
• box {  
  display: inline-block;  
}
```

`display: none` ⇒ No space is reserved

`display: hidden` ⇒ The element becomes invisible but still occupies space.

POSITIONS: This controls where an element is placed on a webpage.

- Types:
- ① Static
 - ② Relative
 - ③ Absolute
 - ④ Fixed
 - ⑤ Sticky

```
• box {  
  position: relative;  
}
```

Static: It is default position of every HTML element.
→ Element appears in normal page flow.
→ top, right, bottom, left do not work.

```
• box {  
  position: static;  
}
```

Relative: The element remains in the normal document flow but moves relative to its original position.
→ The original space is still reserved.

```
• box {  
  position: relative;  
}
```

Absolute: The element is removed from normal document flow and positioned relative to nearest positioned parent

```
• box {  
  position: absolute;  
}
```

Fixed: A fixed element stays in same place when page is scrolled.

Ex: chatbot in webpage

```
• box {  
  position: fixed;  
}
```

Sticky: A sticky element behaves like relative until a specified scroll position is reached, then behaves like fixed.

Ex: Youtube scrolling

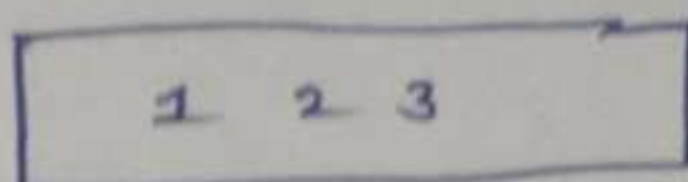
```
• box {  
  position: sticky;  
}
```

FLEX: Flexible Box layout

→ It is used to arrange, align, & distribute space among items inside a container.

```
• box {  
  display: flex;  
}
```

⑥ space-between: Gives exactly equal spacing b/w all items and container edges.



Align-items: It is used to align flex items along the cross axis (vertical axis).

→ Same content as justify-content.

Flex-direction: Controls the direction of flex items

Values:
• row
• row-reverse
• column
• column-reverse

ROW = Dharshu Yuva Divya

COLUMN = Dharshu
Yuva
Divya

GRID: It is a 2-dimensional layout system used to create web page layouts with rows and columns.
→ It allows you to easily place elements in a grid without using float or complex positioning.

Syntax:

```
• container {  
  display: grid;  
}
```

Justify-content: used to align flex items along the main axis (horizontal axis).

Enr justify-content: value

① Flex-start:

→ Places all items at beginning.

justify-content: flex-start

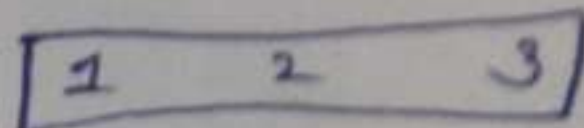
* Website logos starts from left.

② Flex-end:

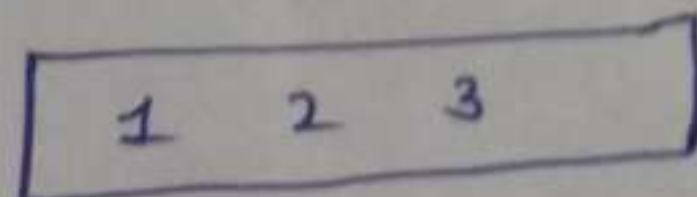
places all items end of container.

③ Flex-center:

④ space-between: places the first item at start, the last item at end, and distributes the remaining space b/w items.



⑤ space-around: Given equal space around every item. The spaces at container edges are half the size of spaces b/w items.



Equal columns: Equal columns of boxes

grid-template-columns: 1fr 1fr 1fr

repeats

grid-template-columns: repeat(3, 1fr);

Pseudo-classes: used to define special state of HTML element.

① Hover: Applies styles when mouse pointer is placed over an element.
Enr: Button color changes.

② Active: Applies styles while element is being clicked. Enr: when u press button color changes.

③ first-child: selects first child element of its parent.

④ Focus: Applies styles when input field is selected.

Enr: HTML
CSS
JavaScript
Java

last-child: HTML
CSS
JavaScript
Java

Pseudo-elements: It is used to style a specific part of an element or insert virtual content before or after an element without modifying HTML.

Before → Add icons before menu items

After → Add ✓ ✖ after text.

first-letter → Newspaper articles (first letter will be highlighted)

first-line → first line will be highlighted.

placeholder → styles text selected by user using mouse

do reverse → selection → placeholder text changes style.